

P2400R1

Library Evolution Report:

2021-02-23 to 2021-05-25

Published Proposal, 2021-06-08

Authors:

[Bryce Adelstein Lelbach \(he/him/his\) — Library Evolution Chair](#) (NVIDIA)
[Fabio Fracassi — Library Evolution Vice Chair](#) (CODE University of Applied Sciences)
[Ben Craig — Library Evolution Vice Chair](#) (NI)
[Billy Baker — Library Incubator Chair](#) (FlightSafety)
[Nevin Liber — Library Incubator Vice Chair](#) (Argonne National Laboratory)
[Inbal Levi — Ranges Chair \(SG9\) and Library Mailing List Review Manager](#)
[Corentin Jabot — Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

1	Execution
1.1	Telecons
1.2	Mailing List Reviews
1.3	Paper Queues and Backlog
1.4	Electronic Polling
1.5	Prioritization
1.6	Chair Guide
1.7	Staff
2	Technical

- 2.1 Future of Library Technical Specifications
- 2.2 Plenary Approved Priorities
 - 2.2.1 Executors
 - 2.2.2 Coroutines Library Support
 - 2.2.3 Networking
 - 2.2.4 Standard Library Modules
- 2.3 Stability
 - 2.3.1 Backports
 - 2.3.2 Interfaces
- 2.4 Other Highlights
 - 2.4.1 Ranges
 - 2.4.2 Formatting and Printing
 - 2.4.3 Text and Unicode
 - 2.4.4 `expected`
 - 2.4.5 `constexpr <cmath>`
 - 2.4.6 Linear Algebra and `mdspan`
- 2.5 Papers Reviewed

This report summarizes the activities of C++'s Library Evolution group from 2021-02-23 to 2021-05-25. It is split into two sections; one focused on *execution* (process, logistics, and operations) and one focused on *technical* work (papers we processed, etc).

Readers are encouraged to also look at:

- [P2145R1](#): Evolving C++ Remotely
- [P2138R3](#): Rules of Design $\Leftrightarrow$ Specification Engagement
- [P2195R0](#): Electronic Straw Polls
- [P0592R4](#): Plan and Priorities for C++23
- [P2289R0](#): 2021 Winter Library Evolution Polls

Prior Library Evolution reports:

- [2020-04-06 to 2020-10-22](#)
- [2020-10-27 to 2021-02-16](#)

§ 1. Execution

§ 1.1. Telecons

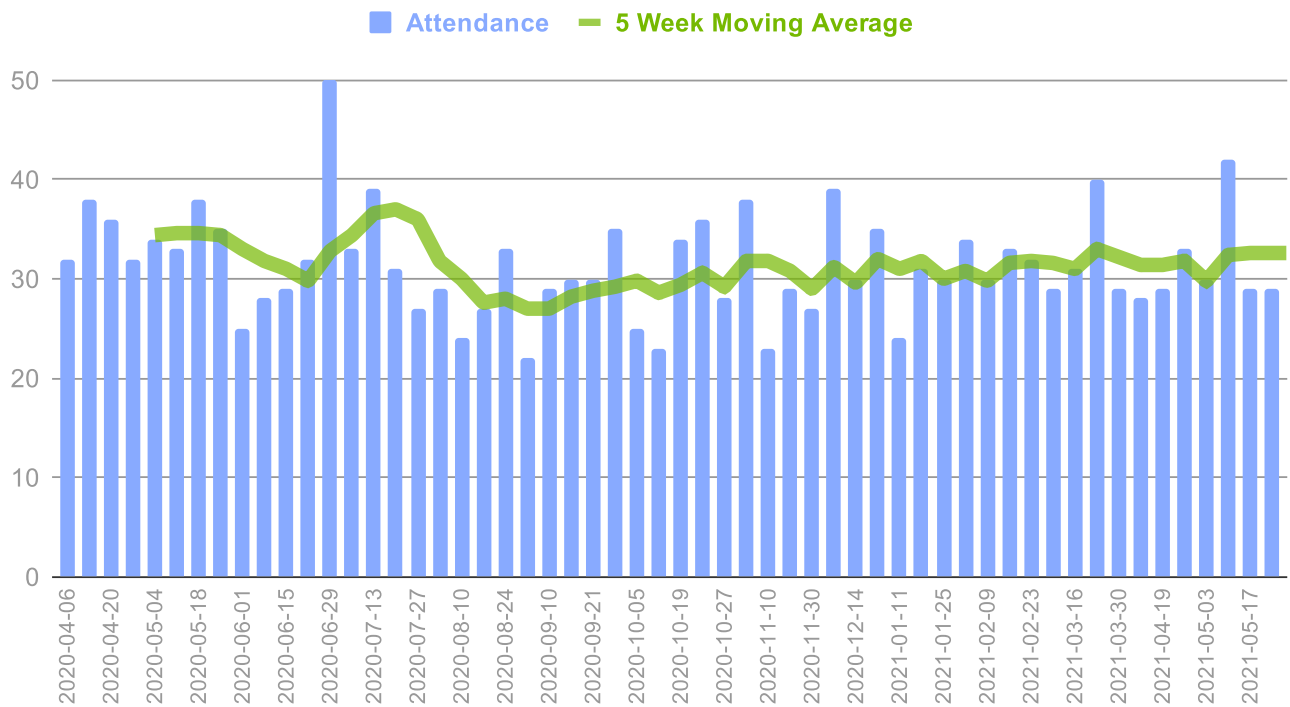
Since 2020-04, Library Evolution has held a 90 minute telecon twice a week to review papers. Typically, 1 or 2 papers are reviewed at each telecon.

The agenda for upcoming telecon reviews can be found [here](#). For details on how to participate in telecon reviews, please view the calendar on documents.isocpp.org or [contact Bryce Adelstein Lelbach](#).

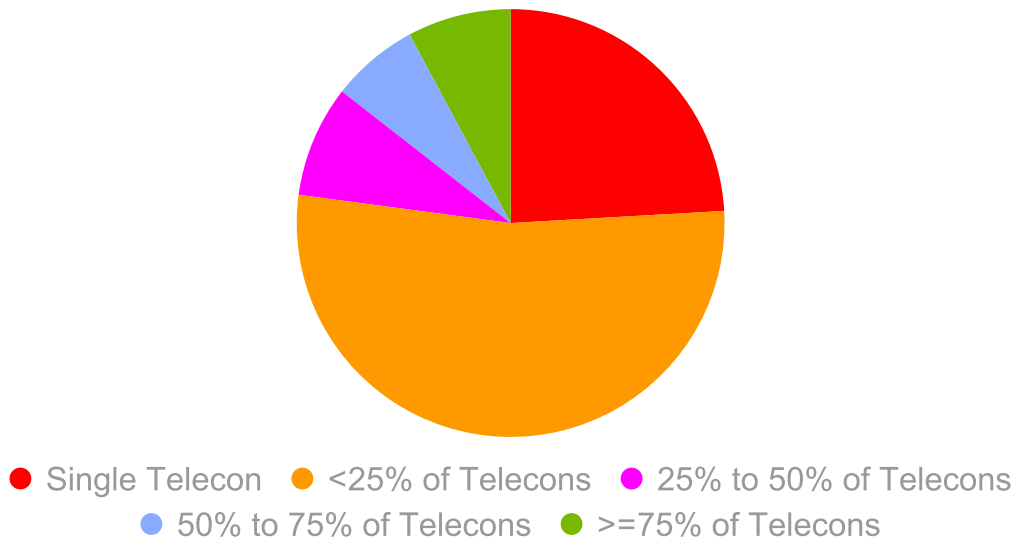
More details on how we use telecons can be found in [P2145R1](#).

	2021-02-23 to 2021-05-25	Since 2020-04-06
# of Telecons	12 + 1 joint with Evolution	56 + 1 joint with Evolution
# of Papers Reviewed On Telecons	20	86
Total # of Attendees	90	166
Mean Telecon Attendance	31.75	31.4
Median Telecon Attendance	29.5	30.5
Mean Telecons Per Attendee	4.23	10.61
Median Telecons Per Attendee	3	4

Library Evolution Telecon Attendance



Library Evolution Telecon Participation Per Attendee



§ 1.2. Mailing List Reviews

In addition to telecons, we review some papers via mailing list discussions.

Mailing list reviews are critical to our operations, as they allow us to increase our paper-processing bandwidth without increasing the frequency or duration of telecons. Additionally, certain classes of papers tend to be

particularly well suited for mailing list reviews, such as new proposals which tend need feedback, not decision making.

We conduct 2 to 3 mailing list reviews concurrently, and each review lasts for a few weeks.

The agenda for upcoming mailing list reviews can be found [here](#). For details on how to participate in mailing list reviews, please [contact Bryce Adelstein Lelbach](#).

More details on how we use mailing lists can be found in [P2145R1](#).

	2021-02-23 to 2021-05-25	Since 2020-04-06
# of Papers Reviewed On The Mailing List	8	23

§ 1.3. Paper Queues and Backlog

Queue	# of Papers
Active	44
Ready for Mailing List Review	3
Scheduled for Mailing List Review	5
Ready for Meeting Review	14
Scheduled for Meeting Review	12
Ready for Electronic Poll	1
Scheduled for Electronic Poll	0

The above information is as of the publication of this paper. For up to date counts and more details on the various queues of papers being handled by Library Evolution, please see [the paper queries GitHub wiki page](#).

§ 1.4. Electronic Polling

Library Evolution conducts a quarterly series of electronic polls to affirm the decisions we make.

The following polling periods have been conducted over the past year:

- 2020 Fall - completed, outcomes can be found in [P2262R0](#).
- 2021 Winter - completed, outcomes can be found in [P2368R0](#).
- 2021 Spring - completed, outcomes can be found in [P2384R0](#).

More details on electronic polling can be found in [P2195R0](#).

§ 1.5. Prioritization

In conjunction with Library, we've prioritized all the papers that have been forwarded to Library, using the priority levels identified in [P0592R4](#) (Plan and Priorities for C++23). See the [prioritized list of papers](#) for details.

§ 1.6. Chair Guide

We have developed a Library Evolution chairing guide, which can be found on the [Library Evolution GitHub wiki](#)

§ 1.7. Staff

Library Evolution is organized by a team, not a single individual:

- Bryce Adelstein Lelbach, Chair
- Fabio Fracassi, Vice Chair
- Ben Craig, Vice Chair
- Billy Baker, Incubator Chair
- Nevin Liber, Incubator Vice Chair
- Corentin Jabot, Mailing List Review Manager
- Inbal Levi, Mailing List Review Manager and Algorithms Task Force Chair

Leadership by a group instead of a team has a number of benefits:

- Increases our capacity and capabilities.
- Reduces the burden on each leader.
- Builds institutional knowledge and redundancy.

We hold a regular staff meeting every two weeks to discuss our operations and make plans for the future.

Additionally, the following noble individuals take minutes for Library Evolution, and deserve accolades:

- Ben Craig
- Inbal Levi
- Mark Hoemmen
- Guy Davidson

§ 2. Technical

§ 2.1. Future of Library Technical Specifications

Over the last decade, we've published a number of Technical Specifications, and we now have a better sense of what is involved in that process and what we get out of it.

There are a few key takeaways:

- Shipping a feature in a Technical Specification does not use less effort than shipping a feature in the International Standard. Roughly the same amount of Library Evolution design effort and Library specification effort is required. We cannot compromise on quality.
- Technical Specifications provide implementation experience, but they do not deliver the levels of usage and deployment experience, or user feedback, that we had wished for.

As such, Library Evolution has become more cautious in its use of Technical Specifications. We seem to have consensus that Technical Specifications have a cost associated with them and should only be used when we have a clear set of questions that we hope to answer through the TS process.

There is also a feeling that Technical Specifications make it harder for us to focus. If we're unsure about something, instead of saying "no" to it, we used to say "let's put it in a TS". However, we now know that putting a facility into a TS uses about the same resources as it would take to put it into the International Standard. As such, we must consider whether we want to be investing our limited time and resources into things that we are not confident in.

Due to the volume of work that is being done on the Standard Library in the International Standard, we have had little time and effort to spare on Technical Specifications in the past few years. Some proposals were forwarded to Technical Specifications as long ago as 2017 and have still not made it to publication.

One of the original motivations for Technical Specifications was to allow the Standard Library to ship new features faster. After the C++11 cycle, which took over a decade, this rationale made sense. However, over the last decade we have adopted a three year ship cycle for the International Standard. Unintuitively, the International Standard is often a faster ship vehicle than Technical Specifications.

This has brought into question the future of general-purpose Technical Specifications, like the Library Fundamentals TS, which collections of possibly unrelated facilities, instead of a focused vehicle with specific goals and questions to be addressed. Library Evolution has not forwarded anything to the Library Fundamentals TS in at least a year.

There are two larger features that were targeting the next revision of the Library Fundamentals TS: [P0323R10](#) (`std::expected`) and [P0009R12](#) (`std::mdspan`). Library Evolution recently reconsidered the ship vehicle for [P0323R10](#) (`std::expected`) and decided to send it to C++23 instead. In 2021-06, Library Evolution will reconsider the ship vehicle for [P0009R12](#) (`std::mdspan`), and it is likely that we will retarget it for C++23 as well. This will leave the Library Fundamentals TS working draft with only a few small additions.

Technical Specifications are not without merit. But they must be focused and used sparingly. They are not a mechanism for us to increase the quantity of things that we ship or the speed at which we ship.

Over the next few months, we need to discuss and formalize how we want to use Technical Specifications in the future, and decide the fate of TSes that are already in the works like the next revision of the Library Fundamentals TS.

§ 2.2. Plenary Approved Priorities

A plenary of the C++ Committee approved [P0592R4](#) (Plan and Priorities for C++23). This section describes the status of the priority Standard Library features described in that plan.

Plenary Approved Priority	Status
Executors	Revision expected in the 2021-06 mailing. Library Evolution review expected in 2021-07.
Coroutines Library Support	Parts under review. Parts need authors and papers.
Networking	At risk for C++23. Unlikely to make it due to dependencies and time constraints.
Standard Library Modules	At risk for C++23. Needs authors and papers.

§ 2.2.1. Executors

In 2020-11, we concluded a series of electronic polls on Executors. The outcomes of those polls can be found in [P2262R0](#).

The Executors authors are in the process of revising [P0443R14](#) according to the outcomes of those polls, which will be published in the 2021-06 mailing. We expect to resume a Library Evolution review of Executors in 2021-07 and to continue that review until the Fall of 2021. At that point, we will make a decision about whether we want to advance the core Executors proposal to Library for C++23.

We had a joint session with Evolution recently to consider [P2279R0](#) (Language mechanism for customization points), an alternative to library-based customization mechanisms such as those used in Executors. We're interested in seeing further exploration of this, but we cannot hold up Library Evolution for it in the meantime.

§ 2.2.2. Coroutines Library Support

A revision of [P2168R1](#) (`std::generator`) was recently received and reviewed multiple times in 2021-02, 2021-03, and 2021-04. The authors have been given thorough feedback and will return with a revision in the coming months.

Otherwise, no further work on the Coroutines Library Support has occurred since Spring 2020 due to a lack of new papers and a lack of revisions of existing papers.

Some of the other work on Coroutines Library Support depends on Executors and may be blocked for now.

§ 2.2.3. Networking

Work on Networking has been proceeding in the Networking Study Group, not Library Evolution. This work depends on Executors, but large parts of it should be able to proceed independent of Executors.

This plenary-approved priority is unlikely to make it into C++23. A substantial Library Evolution design review would be needed prior to merging the proposal, and it is unlikely that we will have the time or the venue to conduct this review before the C++23 feature freeze.

§ 2.2.4. Standard Library Modules

No further work on Standard Library Modules occurred since the Spring 2020 due to a lack of new papers and a lack of revisions of existing papers.

This plenary-approved priority is at risk of missing C++23 unless authors actively drive the work.

§ 2.3. Stability

§ 2.3.1. Backports

Typically, it is very difficult or impossible to make breaking changes to Standard C++ facilities after they have shipped in a particular C++ Standard. However, sometimes there is a small window of opportunity to make such changes.

If no implementation has shipped a particular Standard C++ facility in production, then no one is using the facility yet. Therefore, it is possible for the C++ committee to make what would be a breaking change to said facility in the next C++ Standard without actually causing any breakage. This is only possible if all vendors agree to accept the change and apply it as a backport to their implementation of the older C++ Standard.

Backporting breaking changes to Standard C++ facilities in this way is an extraordinary measure and must only be done with the utmost of care. It should not be considered the norm and we should not assume that we will be able to do it in the future.

When we make these changes, we are effectively changing a C++ Standard after it has shipped. This can cause a great deal of uncertainty in the ecosystem for both users and vendors, because it makes it unclear when a C++ Standard is complete and stable.

Over the past few months, Library Evolution has taken advantage of this window to fix a number of issues impacting ranges and text formatting in C++20:

- [P2372R0](#): Fixing locale handling in chrono formatters
- [P2216R3](#): format improvements
- [P2328R1](#): `join_view` should join all views of ranges
- [P2325R3](#): Views should not be required to be default constructible
- [P2210R2](#): Superior string splitting

The above information is as of the publication of this paper. For up to date information and more details on Library backports [the backports GitHub wikipage](#).

§ 2.3.2. Interfaces

For many years, the C++ committee has been debating how we should balance stability with the need to evolve the Standard Library. At the 2020 Prague meeting, we discussed this in detail - see [P1863R1](#) (ABI - Now or Never) and [P2028R0](#) (What is ABI, and What Should The C++ Committee Do About It?).

It seems unlikely that we'll solve this problem with policy alone, so there is an interest within Library Evolution for exploring technical solutions that would allow us to build library facilities that will be resilient to future changes. Library Evolution recently discussed one such proposal for a language-based solution, [P2123R0](#) (Interfaces: A Facility to Manage ABI/API Evolution). We are interested in seeing more exploration of this space and welcome papers that do additional fact finding and research into this space so that we better understand our needs and how we might benefit from a technical solution like [P2123R0](#).

§ 2.4. Other Highlights

§ 2.4.1. Ranges

We are making significant progress on the objectives laid out in [P2214R0](#) (A Plan for C++23 Ranges).

Over the past few months, we have reviewed the following ranges proposals, some of which make breaking changes from C++20 and are intended as backports:

- [P2325R1](#): Views should not be required to be default constructible (C++20 backport)
- [P2328R0](#): `ranges::join_view` should join all views of ranges (C++20 backport)
- [P2210R2](#): Superior String Splitting (C++20 backport)
- [P2251R0](#): Require `span` & `basic_string_view` to be Trivially Copyable (C++20 backport)
- [P2321R0](#): `views::zip`
- [P2322R2](#): `ranges::fold`

- [P2276R0](#): Fix `std::cbegin()`, `std::ranges::cbegin`, and `cbegin()` for `span`
- [P2302R0](#): Prefer `ranges::contains` over `basic_string_view::contains`
- [P2286R0](#): Formatting Ranges

The following ranges proposals are scheduled for review over the next few months:

- [P2374R0](#): `views::cartesian_product`
- [P2164R4](#): `views::enumerate`
- [P1664R3](#): `ranges::reconstructible_range`
- [P1206R4](#): `ranges::to`
- [P2232R2](#): `ranges::fold`
- [P2286R1](#): Formatting Ranges
- [LWG3534](#): `ranges::set_intersection` and `ranges::set_difference` algorithm requirements are too strict

§ 2.4.2. Formatting and Printing

We began reviewing [P2372R0](#) (Fixing locale handling in chrono formatters), a proposal which aims to make chrono formatters locale-indepenent by default. This proposal is a change from C++20 and we aim to backport it. We will review the next revision in 2021-06.

We’ve continued our work on [P2093R5](#) (Formatted Output), which is being reviewed by both Library Evolution and the Text and Unicode group (SG16). We expect it to be sent to us again over the summer, and hopefully it will be able to advance to Library for C++23 in the next few months.

We’ve also refined [P2286R1](#) (Formatting Ranges) via mailing list to a point where it is ready for review at a Library Evolution telecon, which should happen in the Summer of 2021.

§ 2.4.3. Text and Unicode

The Text and Unicode group (SG16) has forwarded us the next revision of [P1885R5](#) (Naming Text Encodings to Demystify Them), which is a part of the roadmap laid out in [P1238R0](#) (Unicode Direction) and one of their priorities for C++23. We plan to review it over the summer and hopefully advance it to Library.

§ 2.4.4. `std::expected`

We looked at [P0323R10](#) (`std::expected`) to reconsider its ship vehicle. It was originally forwarded to Library in 2017 for the next revision of the Library Fundamentals Technical Specification. We decided that we want to

instead ship it in C++23, as we do not believe the Technical Specification process will improve the proposal and we do not have specific questions for a TS to address.

§ 2.4.5. `constexpr <cmath>`

At the 2021-02 C++ committee plenary meeting, we did not have consensus to advance [P0533R6](#) (`constexpr <cmath>`), due to some open questions and concerns that the semantics were unclear, especially regarding error cases. The paper returned to Library Evolution, where made some design changes that we believe resolves all of the outstanding concerns. The paper will advance to Library pending an electronic poll in Summer 2021.

In 2021-06, Library Evolution will consider the `constexpr`ification of additional parts of `<cmath>`, as discussed in [P1383R0](#) and [P2337R0](#).

§ 2.4.6. Linear Algebra and `mdspan`

We have received a new revision of [P1673R3](#) (BLAS Linear Algebra), which we plan to review in 2021-06. We will also be looking at the latest revision of [P0009R11](#) (`mdspan`) in 2021-06. It was advanced to Library a few years ago, but there are some minor changes we need to review, and we also want to consider retargeting it from the Library Fundamentals TS version 3 to C++23.

§ 2.5. Papers Reviewed

The following papers were reviewed at Library Evolution telecons and advanced to Library Working Group in the 2021 Spring Library Evolution polling period (see [P2384R0](#) for details):

- [P0323R10](#): expected
- [P2325R2](#): Views Should Not Be Required To Be Default Constructible (C++20 backport)
- [P2328R0](#): `ranges::join_view` Should Join All Views Of Ranges (C++20 backport)
- [P2210R2](#): Superior String Splitting (C++20 backport)
- [P2251R1](#): Require `span` & `basic_string_view` To Be Trivially Copyable (C++20 backport)
- [P2321R1](#): `views::zip`
- [P1072R7](#): `basic_string::resize_and_overwrite`
- [P2340R0](#): Clarifying The Status Of The "C Headers"
- [P2301R0](#): Add A `pmr` Alias For `stacktrace`

The following papers were reviewed at Library Evolution telecons:

- [P2279R0](#): Language mechanism for customization points
- [P2123R0](#): Interfaces: A Facility to Manage ABI/API Evolution

- [P2168R1](#): `std::generator`
- [P2322R2](#): `ranges::fold`
- [P2276R0](#): Fix `std::cbegin()`, `std::ranges::cbegin`, and `cbegin()` for `span`
- [P2302R0](#): Prefer `std::ranges::contains` over `std::basic_string_view::contains`
- [P2372R0](#): Fixing locale handling in chrono formatters (C++20 backport)
- [P2093R5](#): Formatted Output
- [P0533R6](#): `constexpr` for `<cmath>`
- [P2273R0](#): `constexpr std::unique_ptr`
- [P0870R4](#): A proposal for a type trait to detect narrowing conversions
- [P1030R4](#): `std::filesystem::path_view`
- [P1072R7](#): `basic_string::resize_and_overwrite`
- [P2301R0](#): Add a `pmr` alias for `stacktrace`
- [P1315R7](#): `memset_explicit`
- [P2340R0](#): Clarifying the status of the "C headers"

The following papers were reviewed on the mailing list:

- [P2286R0](#): Formatting Ranges
- [P2273R0](#): `constexpr unique_ptr`
- [P2283R0](#): `constexpr <memory>` algorithms
- [P2291R0](#): Add `constexpr` Modifiers to Functions `to_chars` and `from_chars` for Integral Types
- [P2255R1](#): A type trait to detect reference binding to temporary
- [P2248R1](#): Enabling list-initialization for algorithms
- [P1068R5](#): Vector API for Random Number Generation
- [P2047R0](#): An allocator-aware `optional` type